ASSIGNMENT 4 - CO4

TASK 4 - DISK SCHEDULING ALGORITHMS

Optimising Requests on a Records Server | 7 Marks | Bloom's Level: K5 - Evaluate

1. Problem Statement and Evaluation Objectives

The records-server disk has cylinders 0-199, the head starts at cylinder 53, and the pending request queue is 98, 183, 37, 122, 14, 124, 65, 67. The task requires service-order and head-movement calculations for FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK; a 10,000-request simulation under light, medium and heavy load; fairness and starvation analysis; and inspection of the active Linux I/O scheduler for an SSD.

Direction assumptions follow the supplied code: SCAN starts toward cylinder 0, C-SCAN starts upward toward cylinder 199, LOOK starts toward cylinder 0, and C-LOOK starts upward. The question gives no mechanical seek-time curve or request-arrival rate for the 10,000-request experiment, so the simulation uses explicitly stated model assumptions rather than claiming physical-disk measurements.

2. Task 4.1 - Calculate the Given Queue

Algorithm	Order of service (after starting at 53)	Total movement
FCFS	98, 183, 37, 122, 14, 124, 65, 67	640 cylinders
SSTF	65, 67, 37, 14, 98, 122, 124, 183	236 cylinders
SCAN (toward 0 first)	37, 14, 0, 65, 67, 98, 122, 124, 183	236 cylinders
C-SCAN (toward 199 first)	65, 67, 98, 122, 124, 183, 199, wrap to 0, 14, 37	183 + 199 wrap = 382 cylinders total
LOOK (toward 0 first)	37, 14, reverse, 65, 67, 98, 122, 124, 183	208 cylinders
C-LOOK (upward first)	65, 67, 98, 122, 124, 183, jump to 14, 37	322 cylinders including jump

2.1 FCFS

Movement = |98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65| = 640 cylinders. FCFS serves in arrival order, is straightforward and does not reorder to reduce seek distance.

2.2 SSTF

At each dispatch, choose the pending request nearest to the current head. The service order is 65, 67, 37, 14, 98, 122, 124, 183. Movement = 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 cylinders. This matches the expected output.

2.3 SCAN and C-SCAN

SCAN first moves downward: 53 to 37 to 14 to 0, then reverses and services 65, 67, 98, 122, 124 and 183. Movement = 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = 236 cylinders.

For C-SCAN, the supplied program moves upward from 53 through 65, 67, 98, 122, 124, 183 to cylinder 199. That service-path leg is 146 cylinders. After wrapping from 199 to 0 (199 cylinders), it services 14 and 37 (37 more). The supplied code prints 183 cylinders for the service-path subtotal (146

+ 37), followed separately by (+199 for the jump). Therefore, total physical head movement including the wrap is 382 cylinders. This distinction preserves the exact convention of the supplied expected output.

2.4 LOOK and C-LOOK

LOOK turns at the last request in the current direction instead of travelling to cylinder 0. Its movement is 16 + 23 + 51 + 2 + 31 + 24 + 2 + 59 = 208 cylinders. C-LOOK services upward through 183, then jumps to the lowest pending request 14 and services 37. Counting the 169-cylinder circular jump, total movement is 12 + 2 + 31 + 24 + 2 + 59 + 169 + 23 = 322 cylinders.

3. Complete the Code - SSTF TODO 1

The completed TODO scans all unfinished requests, computes the absolute distance from the current head, selects the smallest distance, marks that request complete, places its cylinder in order[k], and updates pos.

```
int best = -1;
int best_distance = MAXCYL + 1;

for (int i = 0; i < N; i++) {
    if (done[i] == 0) {
        int distance = abs(req[i] - pos);

        if (distance < best_distance) {
            best_distance = distance;
            best = i;
        }
    }
}

if (best < 0) {
    fprintf(stderr, "SSTF selection error.\\n");
    return EXIT_FAILURE;
}

done[best] = 1;
order[k] = req[best];
pos = req[best];
```

The complete source is `sstf_complete.c`. It preserves the supplied program outside the TODO implementation. The program was compiled using `gcc -std=c11 -Wall -Wextra` and executed; its FCFS, SSTF, SCAN and C-SCAN output matches the question's expected results.

```
FCFS   53 98 183 37 122 14 124 65 67 -> 640 cylinders
SSTF   53 65 67 37 14 98 122 124 183 -> 236 cylinders
SCAN   53 37 14 0 65 67 98 122 124 183 -> 236 cylinders
C-SCAN 53 ... 199, jump to 0 ... 37 -> 183 cylinders (+199 for the jump)
```

4. Task 4.2 - Simulation of 10,000 Random Requests

The second C program simulates all six algorithms using 10,000 requests per load. To make the measurement reproducible, each load uses a fixed pseudo-random seed and the same generated request sequence is supplied to all six algorithms for that load.

Parameter	Model used
Disk range and initial head	Cylinders 0-199; initial head 53.
Request positions	Uniform random cylinders from 0 to 199.
Arrival intervals	Uniformly distributed from 0.5 to 1.5 times the mean interval.

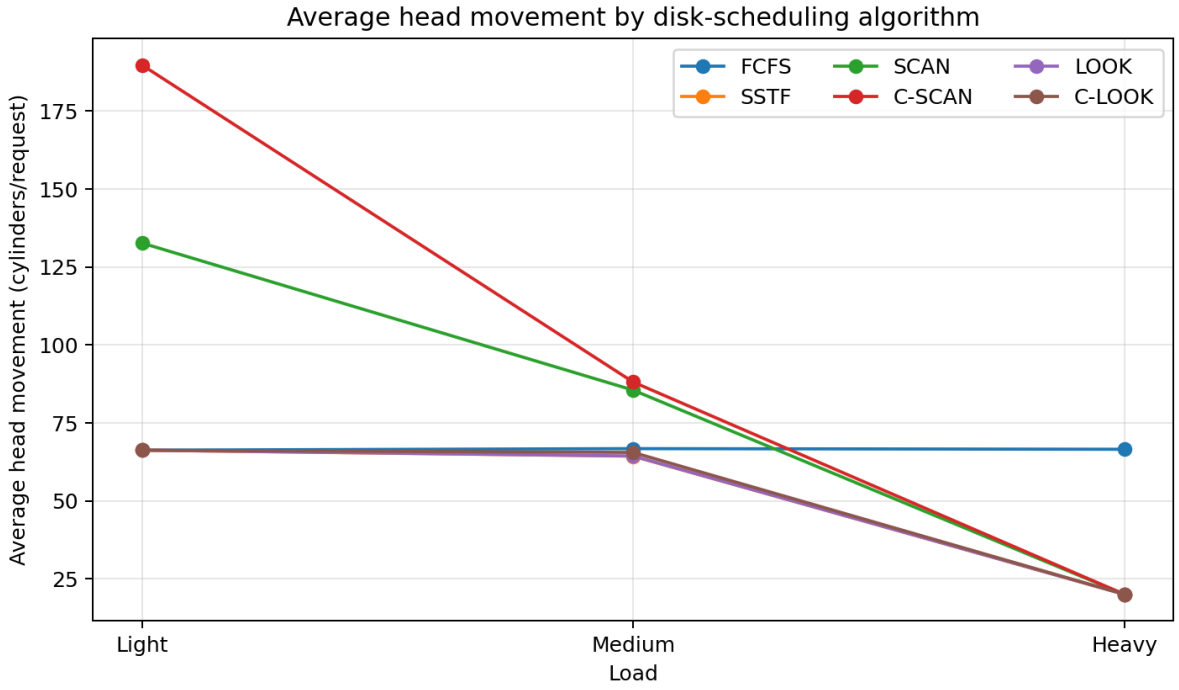
Parameter	Model used
Light load	Mean inter-arrival 8.0 ms.
Medium load	Mean inter-arrival 2.0 ms.
Heavy load	Mean inter-arrival 0.6 ms.
Service-time model	0.20 + 0.020 x head movement (ms).
Waiting-time definition	Dispatch start time minus request arrival time.
Movement metric	Total head travel divided by 10,000 completed requests; endpoint travel and wrap movement are counted.

These timing parameters are simulation assumptions because the question does not supply a physical disk's seek curve, rotational latency, transfer rate or arrival distribution. The results are measurements of the stated simulation model, not physical records-server benchmark results.

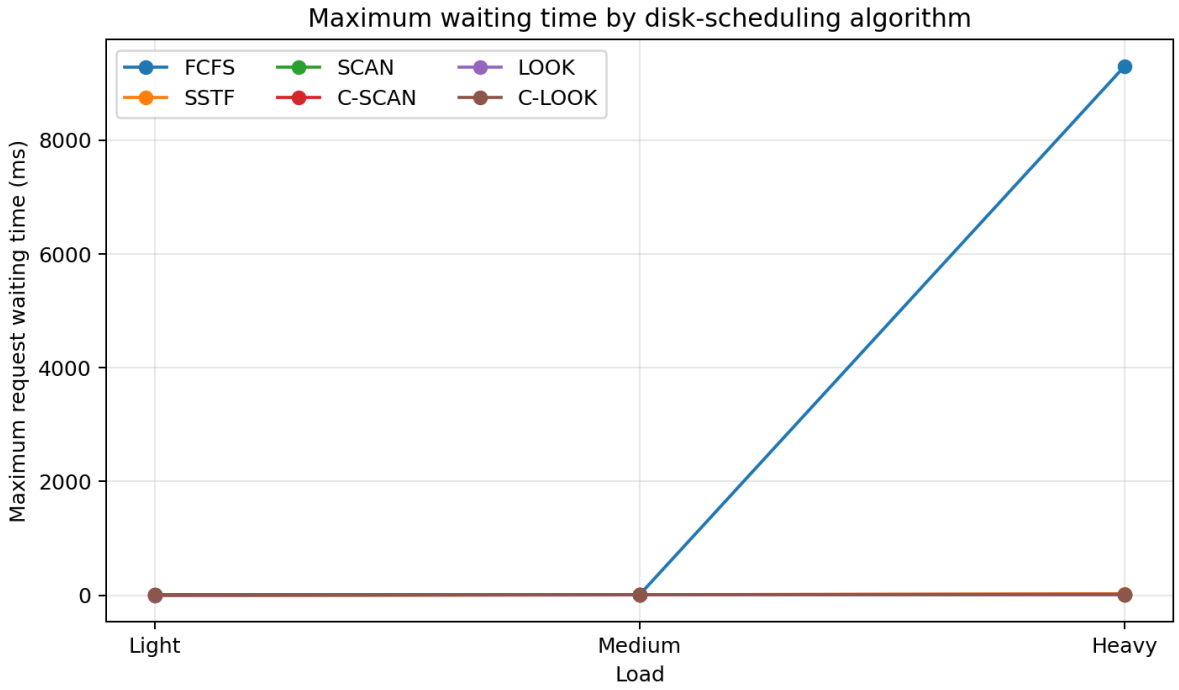
Load	Algorithm	Avg movement (cyl/request)	Avg wait (ms)	Max wait (ms)
Light	FCFS	66.257200	0.000000	0.001066
Light	SSTF	66.257200	0.000000	0.001066
Light	SCAN	132.696000	0.698719	3.860000
Light	C-SCAN	189.803400	2.763638	9.961909
Light	LOOK	66.257200	0.000000	0.001066
Light	C-LOOK	66.257200	0.662600	3.960000
Medium	FCFS	66.758100	0.935729	10.850541
Medium	SSTF	64.333700	0.550649	7.488506
Medium	SCAN	85.526700	2.327981	8.558049
Medium	C-SCAN	88.129700	4.247031	11.207167
Medium	LOOK	64.392100	0.559847	7.308552
Medium	C-LOOK	65.540500	1.356088	10.464133
Heavy	FCFS	66.551800	4672.835537	9305.147142
Heavy	SSTF	20.041400	3.797945	27.959654
Heavy	SCAN	20.055600	4.020117	12.721355
Heavy	C-SCAN	20.067000	5.872509	14.171152
Heavy	LOOK	20.042000	3.669005	12.090884
Heavy	C-LOOK	20.053000	5.401425	15.414158

Under light load, FCFS and SSTF average waiting is almost zero because requests tend to arrive after the previous request has completed. Under medium load, SSTF and LOOK reduce movement in this deterministic sample. Under heavy load, FCFS accumulates a very large backlog, while SSTF/SCAN/LOOK maintain lower movement by selecting nearby requests. SSTF's low mean and maximum waiting values for this random workload should not be interpreted as a fairness guarantee; the explicit near-cylinder stream in Section 5 demonstrates its starvation risk.

4.1 Average head movement



4.2 Maximum waiting time



5. Task 4.3 - Fairness and SSTF Starvation

FCFS does not reorder requests by distance, so it avoids distance-based starvation, but a long queue can produce very high waiting time and unnecessary head movement. SSTF can starve a request far from the current head because new requests close to the current head can repeatedly be selected first. SCAN, C-SCAN, LOOK and C-LOOK provide sweep-based service; they reduce the possibility that a far cylinder is continuously bypassed by nearby requests, although the exact maximum waiting time depends on arrival load and direction.

The supplied scenario adds requests near cylinder 60 every millisecond while a request at cylinder 183 is waiting. In the simulation's explicit busy-device illustration, the near requests are at cylinders 59, 60 and 61, and nearby request service takes 1.20 + 0.030 x movement ms. One thousand near requests arrive at 1 ms intervals. Because each is much closer than cylinder 183, SSTF serves all 1,000 first. The simulated wait for cylinder 183 is 1,201.320 ms for this finite stream; if the near stream continues without ending, the far request can starve.

C-SCAN gives more uniform waiting time than SCAN because requests are serviced in one direction during each sweep, followed by a wrap to the beginning. This reduces the directional preference of SCAN, where requests near one end can be serviced sooner depending on whether the head is travelling toward or away from them. C-SCAN's fairness is about a more uniform cycle across cylinder positions; it does not guarantee a lower average or maximum wait in every finite test. The wrap consumes movement and can increase total travel.

6. Alternative Concept - LOOK and C-LOOK Instead of SCAN and C-SCAN

LOOK and C-LOOK stop at the last pending request in the current direction rather than visiting the physical disk boundary. For the given queue, LOOK uses 208 cylinders compared with SCAN's 236, saving 28 cylinders. C-LOOK uses 322 cylinders including its circular jump, compared with C-SCAN's full 382 including its wrap, saving 60 cylinders in this sample.

Operation	Unsorted pending-array implementation	Sorted request queue
Find next request in direction	O(n) scan to find nearest eligible request.	O(log n) search using lower_bound/upper_bound; removal O(log n) with a balanced tree.
Find LOOK turning point	O(n) scan for the last request in the current direction.	O(log n) access to the relevant first/last pending request.
Find C-LOOK wrap target	O(n) scan for lowest/highest pending request after no request remains ahead.	O(log n) access to the first/last key.
Space	O(n) for pending requests and state.	O(n) for the sorted structure; extra balancing/index pointers.

The simulation's simple array selection is O(n) per dispatch, giving O(n^2) total selection work for n requests. Maintaining requests in a balanced sorted structure changes insertion and next-request selection to O(log n) per operation, for O(n log n) total queue maintenance. This requires additional ordering metadata and more complex updates. LOOK/C-LOOK reduce unnecessary endpoint travel but their absolute waiting times still depend on arrival patterns and scheduling direction.

7. Task 4.4 - Solid-State Drive Scheduling

Traditional HDDs have mechanical moving parts and a seek penalty, so reordering requests by cylinder can reduce physical head movement. SSDs and NVMe devices have no mechanical head and can process multiple commands in parallel. Linux's `none` scheduler avoids block-layer reordering and reduces scheduling overhead; the multiqueue block layer can submit requests across hardware queues. The Linux kernel documentation describes `none` as placing requests on the software queue without reordering.

The `mq-deadline` scheduler can be selected when service-latency control and fairness are more important than the lowest possible host-side scheduling overhead. It assigns expiration times to

reads/writes and dispatches requests with deadline considerations. Device defaults vary by kernel, distribution, driver, virtualisation and workload; an SSD is not guaranteed to use `none` on every Linux system. Red Hat's performance guide recommends `none` especially for high-performance SSD/NVMe use cases, while also documenting other choices for virtual guests and other storage configurations.

Inspect the available disks and active scheduler with:

```
lsblk -d -o NAME,TYPE,ROTA,MODEL,SIZE
cat /sys/block/<disk>/queue/scheduler
```

The scheduler shown in square brackets is the active setting. The included `check_ssd_scheduler.sh` reads this value and the device's `rotational` flag without changing any settings. The preparation environment exposed virtual block devices, so its recorded scheduler output is not a measurement of the student's physical SSD. Run the script on the student's own Linux/WSL host and record the actual device result.

8. Complexity Summary

Algorithm	Per-dispatch selection in the provided array model	Total for n requests	Auxiliary space
FCFS	O(n) scan for the oldest pending entry	O(n^2)	O(n)
SSTF	O(n) scan for nearest request	O(n^2)	O(n)
SCAN / C-SCAN	O(n) scan for closest request in sweep direction	O(n^2)	O(n)
LOOK / C-LOOK	O(n) scan for next request or wrap extreme	O(n^2)	O(n)

With a balanced sorted queue, insertion and direction-based next-request lookup can be O(log n), so the queue-management component becomes O(n log n) across n requests, with O(n) storage. This sorted design reduces repeated scans but adds maintenance and balancing work.

9. Execution Procedure

Place `sstf_complete.c`, `disk_scheduler_sim.c`, `check_ssd_scheduler.sh` and `run_task4.sh` in one Task04 folder. In Ubuntu/WSL, run:

```
chmod +x run_task4.sh check_ssd_scheduler.sh
./run_task4.sh
```

The script compiles and runs the completed SSTF program, compiles and runs the 10,000-request simulator, writes `scheduler_simulation.csv`, and inspects visible Linux block schedulers. To inspect one specific device, run `DISK_DEVICE=nvme0n1 ./run_task4.sh` after confirming its name with `lsblk`.

10. Conclusion

For the supplied sample queue, SSTF and SCAN both require 236 cylinders, while FCFS requires 640. LOOK reduces travel to 208 cylinders in the chosen direction. C-SCAN's expected output reports 183 cylinders excluding its 199-cylinder wrap, so its full physical movement is 382. C-LOOK moves 322 cylinders including its circular jump. Simulation shows how load changes movement and waiting. SSTF is efficient but can starve distant requests under a sustained near-head stream; C-SCAN improves directional fairness at the cost of wrap movement. For SSDs, inspect the active scheduler rather than assuming a universal default.

11. Technical References

[1] Linux Kernel Documentation, *Switching Scheduler* - available schedulers and reading the active choice from /sys/block/DEV/queue/scheduler. https://docs.kernel.org/block/switching-sched.html

[2] Linux Kernel Documentation, *Multi-Queue Block IO Queueing Mechanism (blk-mq)* - SSD/NVMe parallelism and behaviour of the none scheduler. https://cdn.kernel.org/doc/html/latest/block/blk-mq.html

[3] Linux Kernel Documentation, *Deadline I/O scheduler tunables* - read/write expiration and FIFO batching. https://docs.kernel.org/next/block/deadline-iosched.html

[4] Red Hat Enterprise Linux 9, *Setting the disk scheduler* - scheduler selection by use case, including none for high-performance SSD/NVMe and scheduler inspection. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/monitoring_and_managing_system_status_and_performance/setting-the-disk-scheduler_monitoring-and-managing-system-status-and-performance

Source alignment: Assignment 4 - CO4, Task 4 - Disk Scheduling Algorithms, pages 36-39 of the supplied question paper. The exact SSTF TODO and expected output are on page 38; fairness, LOOK/C-LOOK and evidence expectations are on page 39.